

# Data Structures and Algorithms

## Lab 05 - Doubly and Circular Linked Lists: Implementation and Applications

Instructor: Sidrat-ul-Muntha

---

### READ IT FIRST

Prior to start solving the problems in this assignments, please give full concentration on following points.

1. **WORKING** – This is individual lab. If you are stuck in a problem contact your teacher, but, in mean time start doing next question (don't waste time).
  2. **DEADLINE** – One Week after assigned this.
  3. **SUBMISSION** – This assignment needs to be submitted in a soft copy.
  4. **WHERE TO SUBMIT** – Please visit your LMS.
  5. **HOW TO SUBMIT** – Rename all .java files according to the task number (e.g., Task1.java, Task2.java). Place all required .java files in a .zip file named 000-00-0000\_assignment.zip and submit the ZIP file on the LMS.
- 

### Learning Objectives

By the end of this lab, students will be able to:

- Implement a Doubly Linked List using head and tail pointers.
- Perform insertion, deletion, searching, and updating operations.
- Traverse a Doubly Linked List in forward and backward directions.
- Convert a Doubly Linked List into a Circular Doubly Linked List.
- Apply Circular Doubly Linked Lists to a real-world application.

### Task 1: Doubly Linked List with Tail

Create a class named **DoublyLinkedList** that implements a doubly linked list using both **head** and **tail** pointers.

Each node should contain:

- data
- next
- prev

## Implement the Following Methods

- **addBack(data)** – Add a new node at the end of the list using the tail pointer.
- **deleteFront()** – Delete the first node from the list.
- **deleteBack()** – Delete the last node from the list using the tail pointer.
- **searchNode(data)** – Search for a node containing the given value.
- **deleteNode(data)** – Delete the first node containing the given value.
- **addAfter(targetData, data)** – Add a new node after the node containing targetData.
- **addBefore(targetData, data)** – Add a new node before the node containing targetData.
- **displayForward()** – Display all nodes from head to tail.
- **displayBackward()** – Display all nodes from tail to head.
- **updateNode(oldData, newData)** – Find a node containing oldData and update its value to newData.

## Demonstration

In the **main()** function, create a Doubly Linked List and demonstrate all the above operations.

---

## Task 2: Convert Doubly Linked List into Circular Doubly Linked List

Extend the **DoublyLinkedList** class created in **Task 1** by adding the following method:

### **makeCircular()**

The **makeCircular()** method should convert the existing Doubly Linked List into a Circular Doubly Linked List.

After calling **makeCircular()**:

- **tail->next** should point to **head**.
  - **head->prev** should point to **tail**.
  - The list should no longer contain **NULL** links at the beginning or end.
- 

## Task 3: Music Playlist Using Circular Doubly Linked List

Develop a **Music Playlist** application using a Circular Doubly Linked List.

A music player allows users to add, remove, search, and play songs from a playlist. The user can move **forward** to the next song or **backward** to the previous song.

The music player also provides a **Loop Mode** that can be turned **ON or OFF**.

- When **Loop Mode is ON**, reaching the last song and selecting `playNext()` should move to the first song.
- When **Loop Mode is ON**, reaching the first song and selecting `playPrevious()` should move to the last song.
- When **Loop Mode is OFF**, the playlist should stop at the first or last song instead of continuing to the other end.

## Node Structure

Each node should store:

- **songName**
- **artistName**
- **next**
- **prev**

Implement the Following Operations

1. **addSong(songName, artistName)** – Add a new song to the playlist.
2. **deleteSong(songName)** – Delete a song from the playlist.
3. **searchSong(songName)** – Search for a song in the playlist.
4. **playNext()** – Move to the next song and display it.
5. **playPrevious()** – Move to the previous song and display it.
6. **displayPlaylist()** – Display all songs in forward direction.
7. **displayReversePlaylist()** – Display all songs in backward direction.
8. **toggleLoopMode()** – Turn Loop Mode ON or OFF.

## Loop Mode Behavior

Maintain a variable such as:

bool **loopMode**;

Initially, Loop Mode should be **OFF**.

When Loop Mode is **OFF**:

Song 1 → Song 2 → Song 3 → Song 4 → Song 5

**STOP**

When Loop Mode is **ON**:

Song 1 → Song 2 → Song 3 → Song 4 → Song 5



Therefore:

- At the last song:
  - Loop ON → **playNext()** moves to the first song.
  - Loop OFF → **playNext()** remains at the last song and displays an appropriate message.
- At the first song:
  - Loop ON → **playPrevious()** moves to the last song.
  - Loop OFF → **playPrevious()** remains at the first song and displays an appropriate message.

## Demonstration

In the main() function:

1. Create a playlist.
2. Add at least **5 songs**.
3. Display the playlist.
4. Search for a song.
5. Play songs using playNext().
6. Demonstrate that the playlist stops at the last song when Loop Mode is OFF.
7. Turn Loop Mode ON using toggleLoopMode().
8. Demonstrate that playNext() moves from the last song back to the first song.
9. Demonstrate backward navigation using playPrevious().
10. Turn Loop Mode OFF.
11. Demonstrate that backward navigation stops at the first song.
12. Delete a song and display the updated playlist.

**Note:** The purpose of this task is to **apply the Circular Doubly Linked List concept to a real-world music playlist**.

---